

Python Cheat Sheet

Pandas Functions

Import pandas as pd

Category	Function	Description	Example
Data I/O	<code>pd.read_csv(file)</code>	Read CSV file into DataFrame	<code>df = pd.read_csv('data.csv')</code>
	<code>pd.read_excel(file)</code>	Read Excel file	<code>pd.read_excel('data.xlsx')</code>
	<code>df.to_csv(file)</code>	Write DataFrame to CSV	<code>df.to_csv('out.csv', index=False)</code>
	<code>df.to_excel(file)</code>	Write DataFrame to Excel	<code>df.to_excel('out.xlsx')</code>
Create Data	<code>pd.DataFrame(data)</code>	Create DataFrame from dict/list	<code>pd.DataFrame({'A': [1, 2]})</code>
	<code>pd.Series(data)</code>	Create Series	<code>pd.Series([10, 20, 30])</code>
Inspect Data	<code>df.head(n)</code>	First n rows	<code>df.head(3)</code>
	<code>df.tail(n)</code>	Last n rows	<code>df.tail(3)</code>
	<code>df.info()</code>	Column info & types	<code>df.info()</code>
	<code>df.describe()</code>	Summary statistics	<code>df.describe()</code>
	<code>df.shape</code>	Rows, columns count	<code>df.shape</code>
Select & Filter	<code>df['col']</code>	Select column	<code>df['A']</code>
	<code>df[['c1', 'c2']]</code>	Select multiple columns	<code>df[['A', 'B']]</code>
	<code>df.loc[row_labels, col_labels]</code>	Label-based select	<code>df.loc[0:2, 'A':'B']</code>
	<code>df.iloc[row_idx, col_idx]</code>	Index-based select	<code>df.iloc[0:3, 0:2]</code>

Category	Function	Description	Example
	<code>df.query(expr)</code>	Filter rows by expression	<code>df.query("A > 5")</code>
Missing Data	<code>df.isna()</code>	Detect missing values	<code>df.isna()</code>
	<code>df.notna()</code>	Detect non-missing	<code>df.notna()</code>
	<code>df.fillna(val)</code>	Fill missing values	<code>df.fillna(0)</code>
	<code>df.dropna()</code>	Drop rows with NaN	<code>df.dropna()</code>
Sorting & Duplicates	<code>df.sort_values(col)</code>	Sort by column	<code>df.sort_values('A')</code>
	<code>df.sort_index()</code>	Sort by index	<code>df.sort_index()</code>
	<code>df.drop_duplicates()</code>	Remove duplicate rows	<code>df.drop_duplicates()</code>
	<code>df.nlargest(n, col)</code>	Top n rows by column	<code>df.nlargest(3, 'A')</code>
	<code>df.nsmallest(n, col)</code>	Bottom n rows by column	<code>df.nsmallest(3, 'A')</code>
Aggregation & Grouping	<code>df.groupby(col)</code>	Group by column	<code>df.groupby('A')</code>
	<code>.agg(func)</code>	Aggregate with function	<code>df.groupby('A').agg('mean')</code>
	<code>.sum(), .mean()</code>	Aggregation shortcuts	<code>df['A'].mean()</code>
	<code>.count()</code>	Count non-missing	<code>df.count()</code>
Modify Columns	<code>df['new'] = ...</code>	Create/assign column	<code>df['C'] = df['A'] + df['B']</code>
	<code>df.assign(new=...)</code>	Create column (chainable)	<code>df.assign(D=lambda x: x.A*2)</code>
	<code>df.rename(columns={})</code>	Rename columns	<code>df.rename(columns={'A': 'Alpha'})</code>
Merge & Join	<code>pd.concat([df1, df2])</code>	Combine DataFrames	<code>pd.concat([df1, df2])</code>

Category	Function	Description	Example
	<code>df.merge(df2, on='col')</code>	SQL-style join	<code>df.merge(df2, on='ID')</code>
	<code>df.join(df2)</code>	Join on index	<code>df1.join(df2)</code>
Pivot & Reshape	<code>df.pivot(index, columns, values)</code>	Pivot table	<code>df.pivot('date', 'item', 'sales')</code>
	<code>df.melt()</code>	Unpivot table	<code>df.melt(id_vars='A')</code>
	<code>df.stack()</code> / <code>df.unstack()</code>	Stack/unstack levels	<code>df.stack()</code>
	<code>df.T</code>	Transpose	<code>df.T</code>
Date/Time	<code>pd.to_datetime(col)</code>	Convert to datetime	<code>pd.to_datetime(df['date'])</code>
	<code>df['date'].dt.year</code>	Extract year	<code>df['date'].dt.year</code>
	<code>df['date'].dt.month</code>	Extract month	<code>df['date'].dt.month</code>
Export Data	<code>df.to_csv(file)</code>	Export to CSV	<code>df.to_csv('data.csv')</code>
	<code>df.to_excel(file)</code>	Export to Excel	<code>df.to_excel('data.xlsx')</code>

NumPy Functions

Import numpy as np

Category	Function	Description	Example
Array Creation	<code>np.array(list)</code>	Create array from list/tuple	<code>np.array([1, 2, 3])</code>
	<code>np.zeros(shape)</code>	Create array filled with 0s	<code>np.zeros((2,3))</code>
	<code>np.ones(shape)</code>	Create array filled with 1s	<code>np.ones((3,3))</code>
	<code>np.full(shape, val)</code>	Create array filled with given value	<code>np.full((2,2), 7)</code>
	<code>np.eye(n)</code>	Identity matrix	<code>np.eye(4)</code>
	<code>np.arange(start, stop, step)</code>	Evenly spaced values	<code>np.arange(0, 10, 2)</code>

Category	Function	Description	Example
	<code>np.linspace(start, stop, num)</code>	Evenly spaced values (incl. stop)	<code>np.linspace(0, 1, 5)</code>
	<code>np.random.random(size)</code>	Random floats [0,1)	<code>np.random.random((2,2))</code>
	<code>np.random.randint(low, high, size)</code>	Random integers	<code>np.random.randint(0, 10, (3,3))</code>
Array Info	<code>.shape</code>	Dimensions of array	<code>a.shape</code>
	<code>.ndim</code>	Number of dimensions	<code>a.ndim</code>
	<code>.dtype</code>	Data type of elements	<code>a.dtype</code>
	<code>.size</code>	Total number of elements	<code>a.size</code>
Reshaping & Transpose	<code>np.reshape(a, shape)</code>	Change shape	<code>np.reshape(a, (4,3))</code>
	<code>.T</code>	Transpose array	<code>a.T</code>
	<code>np.transpose(a)</code>	Rearrange axes	<code>np.transpose(a)</code>
Math Operations	<code>np.add(a, b)</code>	Element-wise addition	<code>np.add([1,2], [3,4])</code>
	<code>np.subtract(a, b)</code>	Element-wise subtraction	<code>np.subtract([3,4], [1,2])</code>
	<code>np.multiply(a, b)</code>	Element-wise multiplication	<code>np.multiply([2,3], [4,5])</code>
	<code>np.divide(a, b)</code>	Element-wise division	<code>np.divide([4,9], [2,3])</code>
	<code>np.power(a, n)</code>	Power of elements	<code>np.power([2,3], 3)</code>
	<code>np.sqrt(a)</code>	Square root	<code>np.sqrt([4,9])</code>
	<code>np.exp(a)</code>	Exponential	<code>np.exp([1,2])</code>
	<code>np.log(a)</code>	Natural log	<code>np.log([1, np.e])</code>
Statistics	<code>np.sum(a, axis)</code>	Sum along axis	<code>np.sum(a, axis=0)</code>
	<code>np.min(a) / np.max(a)</code>	Min/Max value	<code>np.max(a)</code>
	<code>np.mean(a)</code>	Mean	<code>np.mean(a)</code>
	<code>np.median(a)</code>	Median	<code>np.median(a)</code>
	<code>np.std(a)</code>	Standard deviation	<code>np.std(a)</code>

Category	Function	Description	Example
	<code>np.var(a)</code>	Variance	<code>np.var(a)</code>
	<code>np.cumsum(a)</code>	Cumulative sum	<code>np.cumsum([1,2,3])</code>
	<code>np.cumprod(a)</code>	Cumulative product	<code>np.cumprod([1,2,3])</code>
	<code>np.argmax(a) / np.argmin(a)</code>	Index of max/min	<code>np.argmax(a)</code>
Logic & Comparison	<code>a > val</code>	Element-wise comparison	<code>a[a > 5]</code>
	<code>np.where(cond, x, y)</code>	Conditional selection	<code>np.where(a>5, a, 0)</code>
	<code>np.all(a) / np.any(a)</code>	Check if all/any True	<code>np.any(a > 5)</code>
Sorting & Indexing	<code>np.sort(a)</code>	Sorted array	<code>np.sort(a)</code>
	<code>np.argsort(a)</code>	Indices for sorted order	<code>np.argsort(a)</code>
	Boolean indexing	Filter elements	<code>a[a % 2 == 0]</code>
Concatenation & Split	<code>np.concatenate([a,b], axis)</code>	Join arrays	<code>np.concatenate([a,b], axis=0)</code>
	<code>np.vstack([a,b])</code>	Stack vertically	<code>np.vstack([a,b])</code>
	<code>np.hstack([a,b])</code>	Stack horizontally	<code>np.hstack([a,b])</code>
	<code>np.split(a, n)</code>	Split array	<code>np.split(a, 2)</code>
Linear Algebra	<code>np.dot(a,b) / a @ b</code>	Dot product	<code>a @ b</code>
	<code>np.linalg.inv(a)</code>	Matrix inverse	<code>np.linalg.inv(a)</code>
	<code>np.linalg.solve(a,b)</code>	Solve linear system	<code>np.linalg.solve(a, b)</code>
File I/O	<code>np.save('f', a) / np.load('f.npy')</code>	Save/Load binary	<code>np.save('arr.npy', a)</code>
	<code>np.savetxt('f', a) / np.loadtxt('f')</code>	Save/Load text	<code>np.savetxt('arr.txt', a)</code>

Matplotlib Functions

Import `matplotlib.pyplot` as `plt`

Category	Function	Description	Example
Basic Setup	import matplotlib.pyplot as plt	Import plotting module	import matplotlib.pyplot as plt
	%matplotlib inline	Show plots in Jupyter	%matplotlib inline
Basic Plotting	plt.plot(x, y)	Line plot	plt.plot([1,2,3], [4,5,6])
	plt.scatter(x, y)	Scatter plot	plt.scatter(x, y)
	plt.bar(x, height)	Vertical bar chart	plt.bar(['A','B'], [3,5])
	plt.barh(y, width)	Horizontal bar chart	plt.barh(['A','B'], [3,5])
	plt.hist(data, bins)	Histogram	plt.hist(data, bins=10)
	plt.pie(sizes, labels)	Pie chart	plt.pie([30,70], labels=['A','B'])
Figure Control	plt.figure(figsize=(w,h))	Set figure size	plt.figure(figsize=(8,6))
	plt.subplot(r,c,i)	Create subplot grid	plt.subplot(2,2,1)
	fig, ax = plt.subplots()	Object-oriented API	fig, ax = plt.subplots()
Labels & Titles	plt.title('title')	Set title	plt.title('My Plot')
	plt.xlabel('label')	X-axis label	plt.xlabel('Time')
	plt.ylabel('label')	Y-axis label	plt.ylabel('Value')
	plt.suptitle('title')	Figure-level title	plt.suptitle('Overall Title')
Axis Control	plt.xlim(min,max)	Set x-axis range	plt.xlim(0,10)
	plt.ylim(min,max)	Set y-axis range	plt.ylim(0,50)
	plt.xticks(ticks, labels)	Set x-axis ticks/labels	plt.xticks([0,1], ['Low', 'High'])
	plt.yticks(ticks, labels)	Set y-axis ticks/labels	plt.yticks([0,1], ['Low', 'High'])
Style & Colors	color='r'	Set color	plt.plot(x, y, color='red')
	linestyle='--'	Set line style	plt.plot(x, y, linestyle='--')
	marker='o'	Markers at points	plt.plot(x, y, marker='o')

Category	Function	Description	Example
	<code>plt.style.use('style')</code>	Use predefined style	<code>plt.style.use('ggplot')</code>
Legends	<code>plt.legend()</code>	Show legend	<code>plt.legend(['A', 'B'])</code>
	<code>label='name'</code>	Label for legend	<code>plt.plot(x, y, label='Line 1')</code>
Annotations	<code>plt.annotate(text, xy)</code>	Annotate point	<code>plt.annotate('Peak', xy=(2, 5))</code>
	<code>plt.text(x, y, text)</code>	Add text at location	<code>plt.text(1, 4, 'Note')</code>
Grids	<code>plt.grid(True)</code>	Show grid	<code>plt.grid(True)</code>
Saving Plots	<code>plt.savefig('file.png')</code>	Save figure to file	<code>plt.savefig('plot.png', dpi=300)</code>
Show Plot	<code>plt.show()</code>	Display plot	<code>plt.show()</code>

Seaborn Functions

Import seaborn as sns

Category	Function	Description	Example
Basic Setup	<code>import seaborn as sns</code>	Import Seaborn	<code>import seaborn as sns</code>
	<code>sns.set()</code>	Apply default Seaborn style	<code>sns.set()</code>
	<code>sns.set_theme(style, palette)</code>	Set theme & palette	<code>sns.set_theme(style="whitegrid", palette="pastel")</code>
Relational Plots	<code>sns.relplot(data, x, y, kind)</code>	General relational plot (line/scatter)	<code>sns.relplot(data=df, x="time", y="value", kind="line")</code>
	<code>sns.scatterplot(data, x, y)</code>	Scatter plot	<code>sns.scatterplot(data=df, x="time", y="value")</code>
	<code>sns.lineplot(data, x, y)</code>	Line plot	<code>sns.lineplot(data=df, x="time", y="value")</code>
Categorical Plots	<code>sns.catplot(data, x, y, kind)</code>	General categorical plot	<code>sns.catplot(data=df, x="day", y="total_bill", kind="bar")</code>
	<code>sns.barplot(data, x, y)</code>	Bar chart with error bars	<code>sns.barplot(data=df, x="day", y="total_bill")</code>
	<code>sns.countplot(data, x)</code>	Count of categories	<code>sns.countplot(data=df, x="day")</code>

Category	Function	Description	Example
	<code>sns.boxplot(data, x, y)</code>	Box plot	<code>sns.boxplot(data=df, x="day", y="total_bill")</code>
	<code>sns.violinplot(data, x, y)</code>	Violin plot	<code>sns.violinplot(data=df, x="day", y="total_bill")</code>
	<code>sns.stripplot(data, x, y)</code>	Strip (scatter) plot for categories	<code>sns.stripplot(data=df, x="day", y="total_bill")</code>
	<code>sns.swarmplot(data, x, y)</code>	Non-overlapping points	<code>sns.swarmplot(data=df, x="day", y="total_bill")</code>
Distribution Plots	<code>sns.histplot(data, x, bins)</code>	Histogram	<code>sns.histplot(data=df, x="total_bill", bins=20)</code>
	<code>sns.kdeplot(data, x)</code>	Kernel Density Estimate	<code>sns.kdeplot(data=df["total_bill"])</code>
	<code>sns.ecdfplot(data, x)</code>	Empirical CDF plot	<code>sns.ecdfplot(data=df["total_bill"])</code>
	<code>sns.displot(data, x, kind)</code>	Distribution plot	<code>sns.displot(data=df, x="total_bill", kind="kde")</code>
Matrix Plots	<code>sns.heatmap(data)</code>	Heatmap of matrix/correlation	<code>sns.heatmap(df.corr(), annot=True)</code>
	<code>sns.clustermap(data)</code>	Clustered heatmap	<code>sns.clustermap(df.corr())</code>
Regression Plots	<code>sns.regplot(data, x, y)</code>	Scatter + regression line	<code>sns.regplot(data=df, x="total_bill", y="tip")</code>
	<code>sns.lmplot(data, x, y)</code>	Linear model plot	<code>sns.lmplot(data=df, x="total_bill", y="tip")</code>
Pair & Joint Plots	<code>sns.pairplot(data)</code>	Pairwise relationships	<code>sns.pairplot(df)</code>
	<code>sns.jointplot(data, x, y, kind)</code>	Joint bivariate plot	<code>sns.jointplot(data=df, x="total_bill", y="tip", kind="hex")</code>
Style & Color Control	<code>sns.set_style(style)</code>	Change style	<code>sns.set_style("whitegrid")</code>
	<code>sns.set_palette(palette)</code>	Change color palette	<code>sns.set_palette("muted")</code>
	<code>sns.color_palette()</code>	Get color list	<code>sns.color_palette("coolwarm", 8)</code>
Saving Plots	<code>plt.savefig('file.png')</code>	Save Seaborn plot	<code>plt.savefig("sns_plot.png", dpi=300)</code>

List Functions

List (Ordered, Mutable, Allows Duplicates)

Operation / Method	Description	Example
Creation	Create with square brackets or <code>list()</code>	<code>lst = [1, 2, 3]</code> <code>lst2 = list((4, 5))</code>
<code>append(x)</code>	Add to end	<code>lst.append(4) → [1, 2, 3, 4]</code>
<code>extend(iterable)</code>	Add multiple elements	<code>lst.extend([5, 6])</code>
<code>insert(i, x)</code>	Insert at index	<code>lst.insert(1, "a")</code>
<code>remove(x)</code>	Remove first occurrence	<code>lst.remove(2)</code>
<code>pop([i])</code>	Remove & return (default last)	<code>lst.pop()</code>
<code>clear()</code>	Remove all	<code>lst.clear()</code>
<code>index(x)</code>	First index of value	<code>lst.index(3)</code>
<code>count(x)</code>	Count occurrences	<code>lst.count(2)</code>
<code>sort()</code>	Sort in place	<code>lst.sort()</code>
<code>reverse()</code>	Reverse in place	<code>lst.reverse()</code>
<code>copy()</code>	Shallow copy	<code>lst2 = lst.copy()</code>

Tuple Functions

Tuple (Ordered, Immutable, Allows Duplicates)

Operation / Method	Description	Example
Creation	Parentheses or <code>tuple()</code>	<code>tup = (1, 2, 3)</code> <code>tup2 = tuple([4, 5])</code>
<code>count(x)</code>	Count occurrences	<code>tup.count(2)</code>
<code>index(x)</code>	First index of value	<code>tup.index(3)</code>
Unpacking	Assign tuple elements	<code>a, b = (1, 2)</code>

Dictionary Functions

Dictionary (Key-Value Pairs, Mutable, Keys Unique)

Operation / Method	Description	Example
Creation	Curly braces or <code>dict()</code>	<code>d = {"a": 1, "b": 2} d2 = dict(c=3)</code>
<code>get(key, default)</code>	Get value, or default if missing	<code>d.get("a", 0)</code>
<code>keys()</code>	All keys	<code>d.keys()</code>
<code>values()</code>	All values	<code>d.values()</code>
<code>items()</code>	All (key, value) pairs	<code>d.items()</code>
<code>update(other)</code>	Merge/update	<code>d.update({"c": 3})</code>
<code>pop(key)</code>	Remove & return	<code>d.pop("a")</code>
<code>popitem()</code>	Remove & return last item	<code>d.popitem()</code>
<code>setdefault(k, v)</code>	Get or set default	<code>d.setdefault("d", 4)</code>
<code>clear()</code>	Remove all	<code>d.clear()</code>
<code>copy()</code>	Shallow copy	<code>d2 = d.copy()</code>
<code>fromkeys(iterable, value)</code>	Create new dict with keys	<code>dict.fromkeys([1, 2], 0)</code>

Set Functions

Set (Unordered, Mutable, No Duplicates)

Operation / Method	Description	Example
Creation	Curly braces or <code>set()</code>	<code>s = {1, 2, 3} s2 = set([4, 5])</code>
<code>add(x)</code>	Add element	<code>s.add(4)</code>
<code>remove(x)</code>	Remove element (error if missing)	<code>s.remove(3)</code>
<code>discard(x)</code>	Remove if present	<code>s.discard(3)</code>
<code>pop()</code>	Remove & return arbitrary	<code>s.pop()</code>
<code>clear()</code>	Remove all	<code>s.clear()</code>
<code>copy()</code>	Shallow copy	<code>s2 = s.copy()</code>
<code>union(other)</code>	All elements from both	<code>s.union({4, 5})</code>
<code>intersection(other)</code>	Common elements	<code>s.intersection({3, 4})</code>
<code>difference(other)</code>	In set but not in other	<code>s.difference({3, 4})</code>
<code>symmetric_difference(other)</code>	In one but not both	<code>s.symmetric_difference({3, 4})</code>
<code>update(other)</code>	Add from other	<code>s.update({4, 5})</code>
<code>intersection_update(other)</code>	Keep only common	<code>s.intersection_update({3, 4})</code>

Operation / Method	Description	Example
<code>difference_update(other)</code>	Remove common	<code>s.difference_update({3,4})</code>
<code>symmetric_difference_update(other)</code>	Update to symmetric diff	<code>s.symmetric_difference_update({3,4})</code>
<code>issubset(other)</code>	Check subset	<code>s.issubset({1,2,3})</code>
<code>issuperset(other)</code>	Check superset	<code>s.issuperset({1,2})</code>
<code>isdisjoint(other)</code>	True if no common	<code>s.isdisjoint({5,6})</code>

Lesser-used but powerful functions

Function	Why Important	Example
<code>enumerate(iterable)</code> <i>(built-in)</i>	Get index + value in loop	<code>for i,v in enumerate(lst): ...</code>
<code>zip(list1, list2)</code> <i>(built-in)</i>	Combine lists element-wise	<code>list(zip([1,2],[3,4])) → [(1,3),(2,4)]</code>
<code>any()</code> / <code>all()</code> <i>(built-in)</i>	Check if any/all elements true	<code>any([0,1,0]) → True</code>
<code>min()</code> / <code>max()</code> / <code>sum()</code>	Quick aggregation	<code>sum([1,2,3]) → 6</code>
List comprehension	One-liner creation	<code>[x**2 for x in range(5)]</code>
<code>sorted()</code> <i>(built-in)</i>	Returns sorted copy	<code>sorted(lst, reverse=True)</code>
Tuple unpacking	Assign multiple vars at once	<code>a,b,c = (1,2,3)</code>
Named tuple (<code>collections.namedtuple</code>)	Tuple with named fields	<code>from collections import namedtuple</code>
Dict comprehension	Create in one line	<code>{x: x**2 for x in range(5)}</code>
<code>dict.get(key, default)</code>	Avoid KeyError	<code>d.get("x", 0)</code>
<code>dict.setdefault()</code>	Get or set default	<code>d.setdefault("x", 1)</code>
<code>collections.defaultdict</code>	Auto-create missing keys	<code>from collections import defaultdict</code>
Merging dicts (<code>`</code>	<code>)</code> (Python 3.9+)	Merge two dicts easily
Set comprehension	Quick creation	<code>{x**2 for x in range(5)}</code>
<code>frozenset()</code>	Immutable set	<code>fs = frozenset([1,2,3])</code>
Chaining multiple operations	Combine in one	<code>a.union(b).intersection(c)</code>

General Built-ins (Work with All)

Function	Description	Example
<code>len()</code>	Length	<code>len(lst)</code>
<code>type()</code>	Type of object	<code>type(d)</code>
<code>id()</code>	Memory address	<code>id(a)</code>
<code>isinstance()</code>	Check type	<code>isinstance(lst, list)</code>
<code>map()</code>	Apply function to iterable	<code>list(map(str, [1,2,3]))</code>
<code>filter()</code>	Keep matching values	<code>list(filter(lambda x: x>2, [1,2,3,4]))</code>
<code>reduce()</code> (<i>from functools</i>)	Cumulative computation	<code>reduce(lambda a,b: a+b, lst)</code>
<code>enumerate()</code>	Index + value	<code>enumerate(lst)</code>
<code>reversed()</code>	Reverse iterator	<code>list(reversed(lst))</code>